in28minutes

Getting Started with Apache Maven

The Scenario: Sharing Your Code

Context: You have written a wonderful library with 100 Java files

Goal: You want your friend to use it

The Question: How do you share it?

- **Option A**: Email/Share the `.java` files? (Messy)
- **Option B**: A standardized compiled package format with .class files? ✅

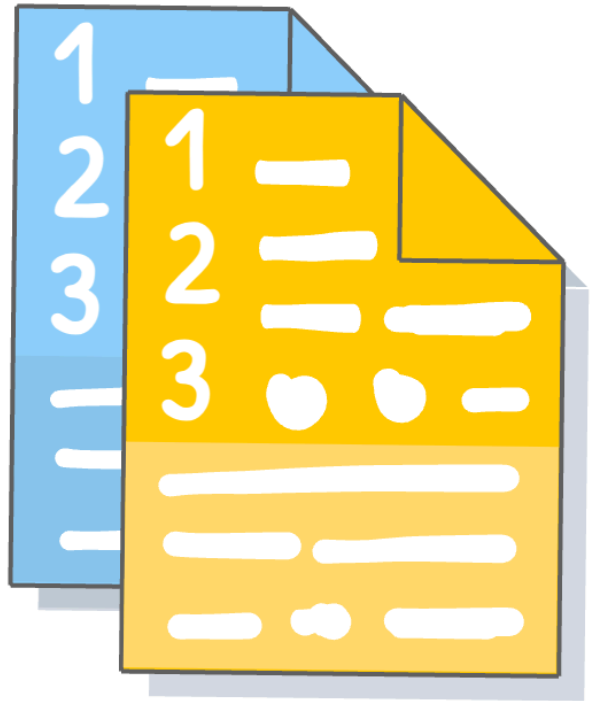
Create a Compiled Java Package

1. **Source Code**: You write `MyClass.java`

2. **Compilation**: You run `javac MyClass.java`
   - Checks for syntax errors
   - Converts Source Code into **Bytecode**
   - Generates `MyClass.class` (Machine/JVM Readable)

Question 1: How to do this for 100s of classes?

Question 2: How to package these classes as one unit?



What is a JAR File?

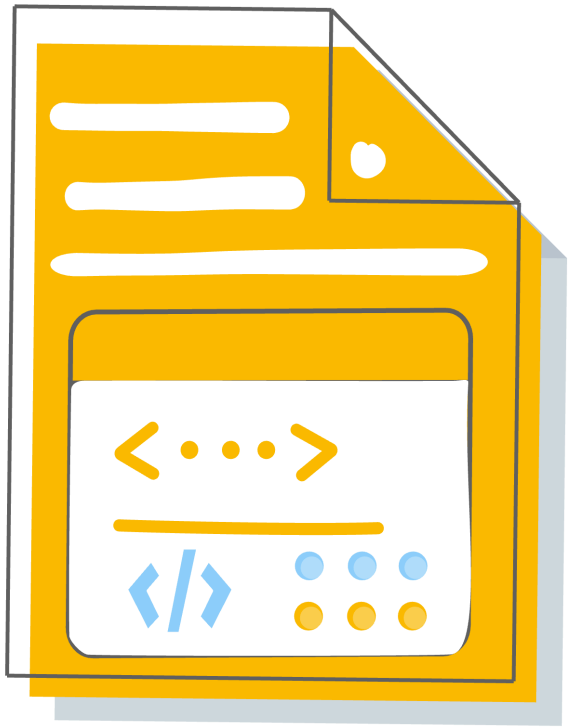
Definition: Java **AR**chive

Analogy: A **ZIP file** for Java classes

Contents:

- Compiled `.class` files
- Metadata (Configuration)
- Resources (Properties, XMLs, Images)

Purpose: The standard way to distribute and reuse Java libraries.



Advantage: Reusing Other Frameworks (e.g., Spring)

Scenario: You want to build a Web Service

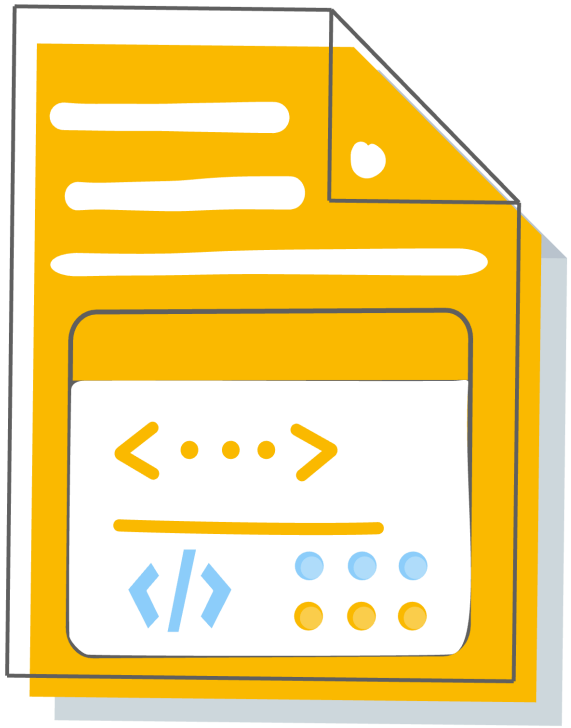
Don't Reinvent the Wheel: Use **Spring MVC**

How?

- Go to Spring website
- Download `spring-core.jar`, `spring-web.jar`, etc
- Copy them into your project

Result: You are "reusing" code via JARs

Is Manual Copying The Best Approach?

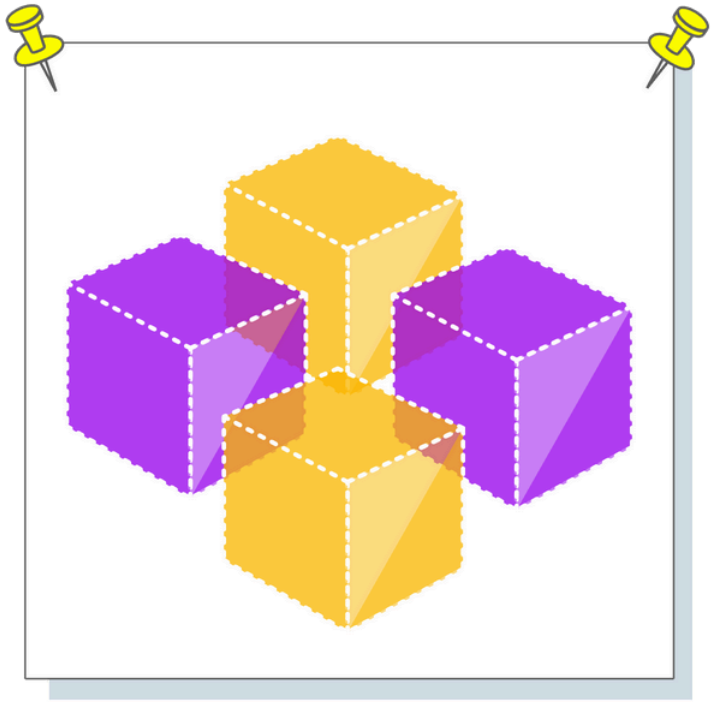


The Nightmare: Dependency Management

The Trap: Dependencies have... *dependencies*

Example:

- You want **Spring MVC**
- Spring MVC needs **Spring Core**
- Spring Core needs **Commons Logging** and ...



The "Manual" Reality: Dependency Management is Complex

- You have to download ALL of them manually
- You have to exact match the **Versions** (JAR Hell)
- One wrong version -> Application crashes at runtime

The Nightmare: Building a Modern Java App

Scenario: Imagine building modern Java app

The Scale: Very Complex

- 100,000 Java Classes
- 1 Million Lines of Code
- 150+ External Libraries (Spring, Hibernate, Logging...)

The Problem:

- How to compile 100,000 classes?
- How to download right version of each dependency?
- How to run unit tests automatically?
- How to package the application into a JAR file?

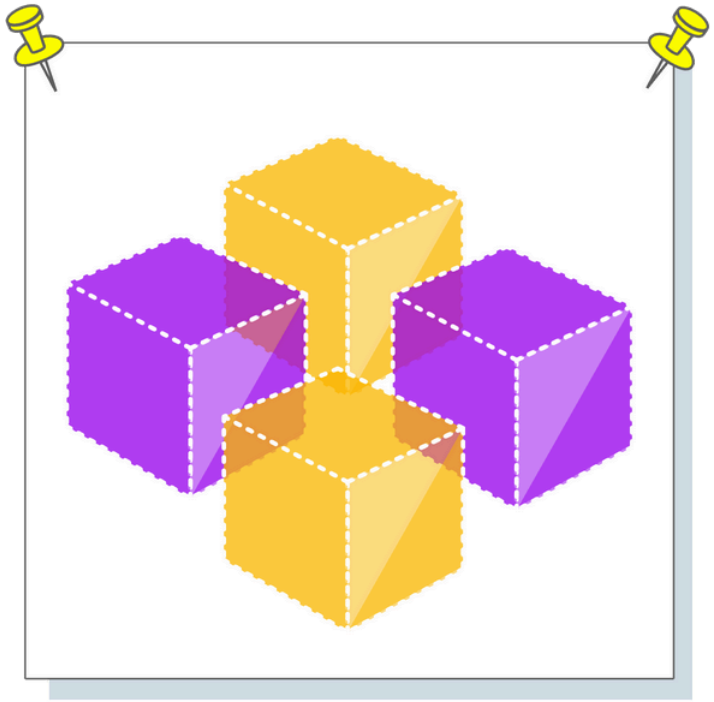
What is Maven?

What is Maven?: The most popular build automation tool for Java

The Magic: Standardizes everything

- **Standard Project Structure**: Code always goes in the same place
- **Standard Dependency Management**: Define *what* you need, Maven gets it
- **Standard Build Lifecycle**: A consistent way to compile, test, and package

Benefit: You focus on code, Maven handles the assembly



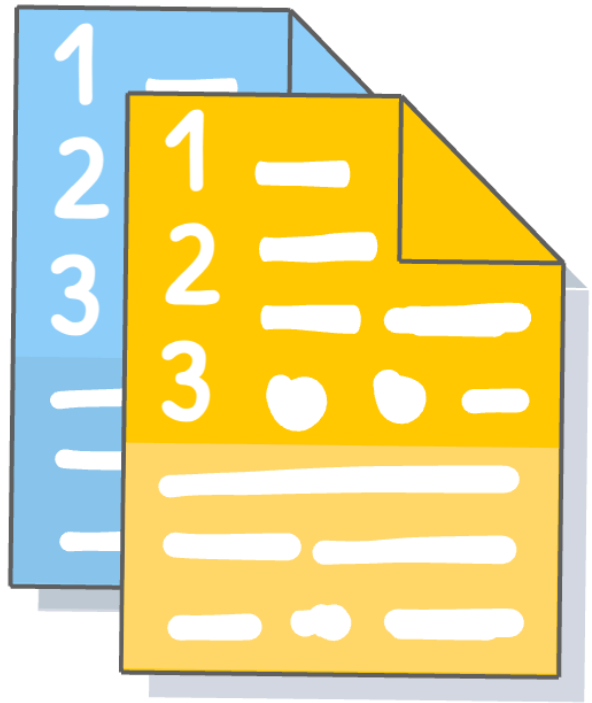
Core Concept 1: The Project Object Model (pom.xml)

The Heart of Maven: Everything is defined in `pom.xml`

Identity: How we identify a project

- **GroupId**: The organization (e.g., `com.in28minutes`) - Like a package name
- **ArtifactId**: The project name (e.g., `learn-maven`) - Like a class name
- **Version**: The specific release (e.g., `1.0.0`)

Why?: This unique ID allows *other* projects to use *your* project as a dependency.



Core Concept 2: Dependency Management

Maven dependencies: Frameworks & libraries used in a project

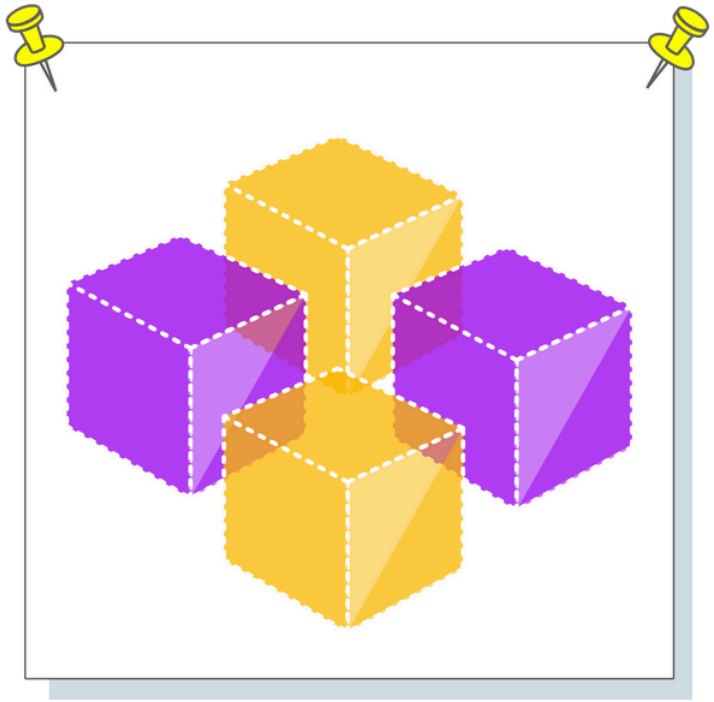
- **Declarative**: You just say *what* you want.
- **Transitive Dependencies**: The Real Power

You ask for `spring-boot-starter-webmvc`

Maven *automatically* downloads:

- Spring MVC, Spring Core, Jackson (JSON), Tomcat, Logging...

No more manual JAR hunting!

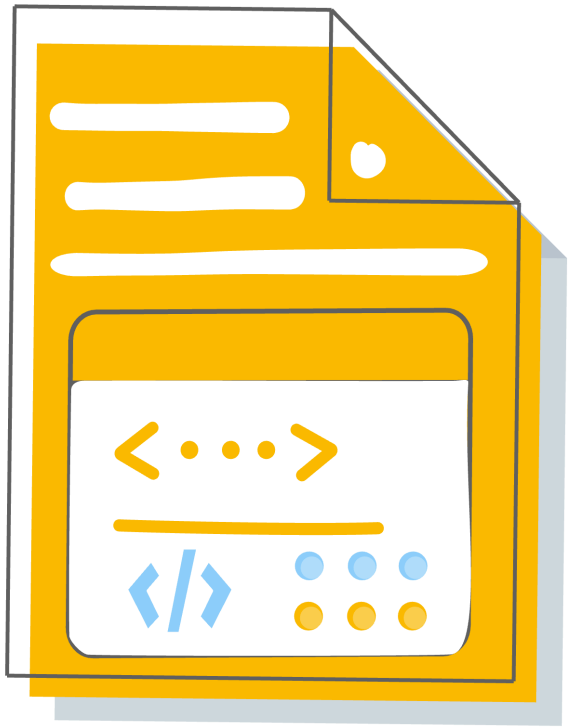


Core Concept 3: Repositories

Where do the JARs come from?

- **1. Central Repository**: The internet's repository of Java libraries and frameworks
  - Maven checks here first for dependencies
- **2. Local Repository**: A folder on *your* machine (`.m2/repository`)
  - Maven stores downloaded JARs here (Caches them)
  - Next time you build, it's fast!

Flow: Request Dependency -> Check Local -> If Missing, Download from Central -> Save to Local



Core Concept 4: Convention Over Configuration

The Rule: Don't invent your own folder structure

Why?:

- Any developer can open any Maven project and know *exactly* where the code is
- No configuration needed to tell the compiler where to find files

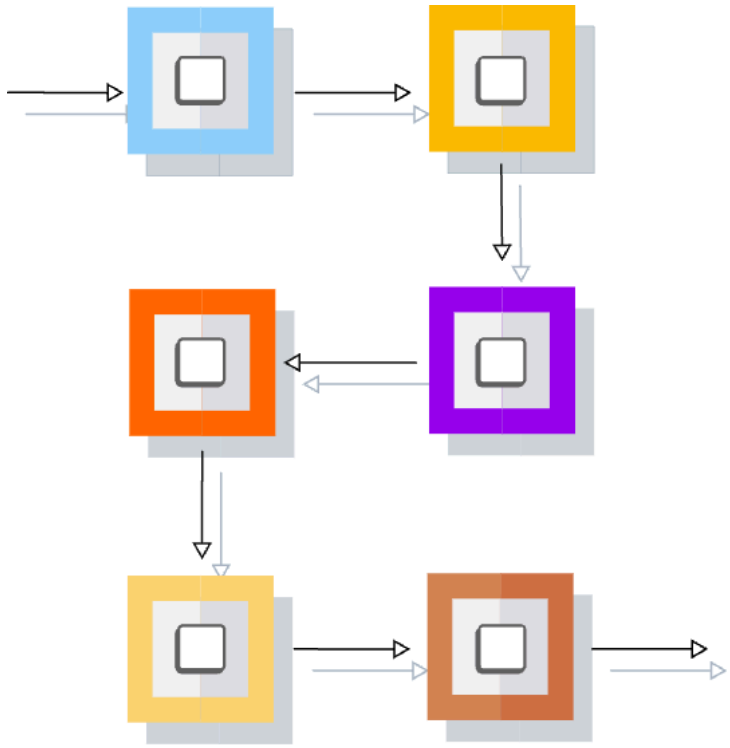
Consistency: Allows IDEs (Eclipse, IntelliJ) to import projects instantly

```
learn-maven
|-- pom.xml
`-- src
    |-- main
    |   |-- java        (Source Code)
    |   `-- resources   (Configuration)
    `-- test
        |-- java        (Unit Test Code)
        `-- resources   (Test Configuration)
```

Core Concept 5: The Build Lifecycle

Sequence: Commands run in a specific order

- `validate`
- `compile`
- `test` (Runs unit tests)
- `package` (Creates JAR)
- `verify` -> `install` -> `deploy`



Rule: Running a phase runs all *previous* phases

- `mvn package`: Validate -> Compile -> Test -> Package
- `mvn test`: Validate -> Compile -> Test

Important Maven Commands

`mvn clean`: Deletes the `target` directory (clean slate)

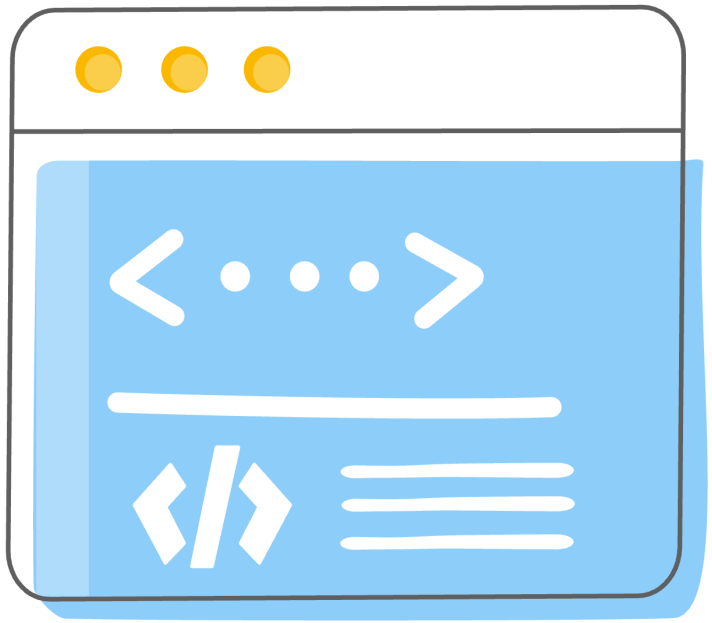
`mvn compile`: Compiles source code

`mvn test`: Runs unit tests

`mvn package`: Creates the JAR file

`mvn install`: Installs the JAR to your *Local Repository* (for other local projects to use)

`mvn help:effective-pom`: See the full calculated configuration



Maven & Spring Boot: The Perfect Pair

Starter Dependencies: One dependency to rule them all

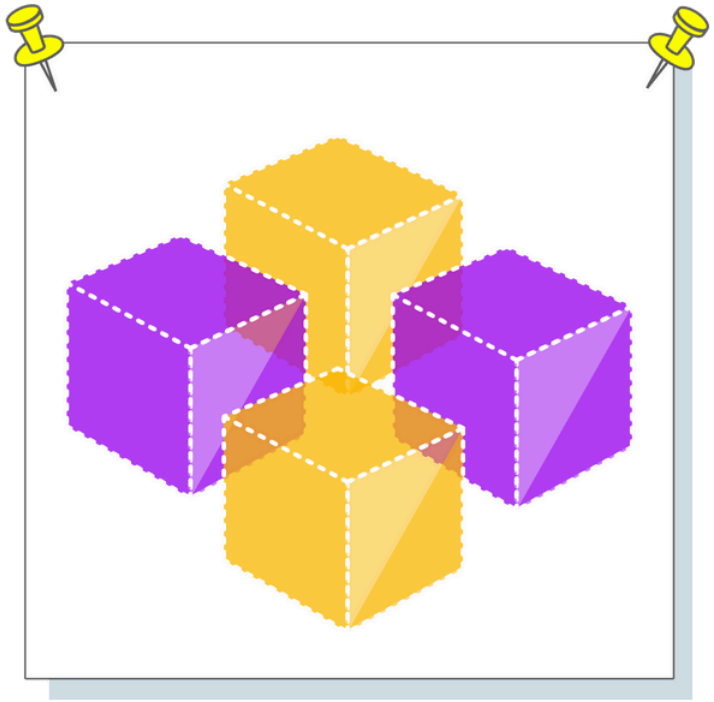
- `spring-boot-starter-webmvc` -> all web dependencies

Parent POM: `spring-boot-starter-parent`

- Manages versions for you (No need to specify versions for common libraries!)

Spring Boot Maven Plugin:

- `mvn spring-boot:run` - Run the app easily
- `mvn spring-boot:build-image` - Create a Container image easily!



Understanding Versioning (Semantic Versioning)

Format: `MAJOR.MINOR.PATCH[-MODIFIER]` (e.g., `10.0.1`)

- **MAJOR**: Lot of work to upgrade (10.0.0 to 11.0.0)
- **MINOR**: Little to no work to upgrade (10.1.0 to 10.2.0)
- **PATCH**: Bug fixes only (10.5.4 to 10.5.5)
- **MODIFIERs**:
  - **SNAPSHOT**: Usage in development (e.g., `10.0.0-SNAPSHOT`) - Unstable
  - **M1/M2/RC1**: Milestones/Release Candidates
  - **Release**: Production ready content

Tip: Avoid SNAPSHOTs in production!

Maven - Summary

Feature	Description
pom.xml	Project Object Model. Defines project identity (GAV), dependencies, and build configuration.
Dependency Management	Automatically downloads libraries and their transitive dependencies.
Repositories	**Local** (Cache on your machine) and **Central** (Internet).
Lifecycle	Standard build steps: *validate* -> *compile* -> *test* -> *package* -> *install*.
Standard Structure	*src/main/java* for code, *src/test/java* for tests.